Logistic Regression and Sentiment analysis

In this assignment you will implement and experiment with various feature engineering techniques in the context of Logistic Regression models for Sentiment classification of movie reviews.

1. Read lexicons of positive and negative sentiment words.
2. Implement overall positive and negative lexicon count features.
3. Implement per-lexicon-word count features.
4. Implement document length feature.
5. Implement deictic features.
6. Analysis of results.
7. Bonus points.

We will use the LR model implemented in sklearn:

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Write Your Name Here: Basit Hussain

Submission Instructions

1. Click the Save button at the top of the Jupyter Notebook/Google Colab.
2. Please make sure to have entered your name above.
3. Select Cell -> All Output -> Clear. This will clear all the outputs from all cells (but will keep the content of ll cells).
4. Select Cell -> Run All. This will run all the cells in order, and will take several minutes.
5. Once you've rerun everything, select File -> Download as/Print -> PDF and download a PDF version *LR-SentimentAnalysis.pdf* showing the code and the output of all cells, and save it in the same folder that contains the notebook file *LR-SentimentAnalysis.ipynb*.
6. Look at the PDF file and make sure all your solutions are there, displayed correctly. The PDF is the only thing we will see when grading!
7. Submit **both** your PDF and notebook on Canvas.
8. Verify your Canvas submission contains the correct files by downloading it after posting it on Canvas.

From documents to feature vectors

This section illustratess the prototypical components of machine learning pipeline for an NLP task, in this case document classification:

1. Read document examples (train, devel, test) from files with a predefined format:
    - assume one document per line, usign the format "<label> <text>".
2. Tokenize each document:
    - using a spaCy tokenizer.
3. Feature extractors:
    - so far, just words.
4. Process each document into a feature vector:
    - map document to a dictionary of feature names.
    - map feature names to unique feature IDs.
    - each document is a feature vector, where each feature ID is mapped to a feature value (e.g. word occurences).

```
1 import spacy
2 from spacy.lang.en import English
3 from scipy import sparse
4 from sklearn.linear_model import LogisticRegression
```

```
1 # Create spaCy tokenizer.
2 spacy_nlp = English()
3
4 def spacy_tokenizer(text):
5     tokens = spacy_nlp.tokenizer(text)
```

```
    return [token.text for token in tokens]

```

```
def read_examples(filename):
    X = []
    Y = []
    with open(filename, mode='r', encoding='utf-8') as file:
        for line in file:
            [label, text] = line.rstrip().split(' ', maxsplit=1)
            X.append(text)
            Y.append(label)
    return X, Y

```

```
def word_features(tokens):
    feats = {}
    for word in tokens:
        feat = 'WORD_%s' % word
        if feat in feats:
            feats[feat] += 1
        else:
            feats[feat] = 1
    return feats
```

```
def add_features(feats, new_feats):
    for feat in new_feats:
        if feat in feats:
            feats[feat] += new_feats[feat]
        else:
            feats[feat] = new_feats[feat]
    return feats
```

This function tokenizes the document, runs all the feature extractors on it and assembles the extracted features into a dictionary mapping feature names to feature values. It is important that feature names do not conflict with each other, i.e. **different features should have different names**. Each document will have its own dictionary of features and their values.

```
def docs2features(trainX, feature_functions, tokenizer):
    examples = []
    count = 0
    for doc in trainX:
        feats = {}
        tokens = tokenizer(doc)
        for func in feature_functions:
            add_features(feats, func(tokens))
        examples.append(feats)
        count += 1
        if count % 100 == 0:
            print('Processed %d examples into features' % len(examples))
    return examples

```

```
# This helper function converts feature names to unique numerical IDs.

def create_vocab(examples):
    feature_vocab = {}
    idx = 0
    for example in examples:
        for feat in example:
            if feat not in feature_vocab:
                feature_vocab[feat] = idx
                idx += 1
    return feature_vocab

```

```
# This helper function converts a set of examples from a dictionary of feature names to values representation
# to a sparse representation of feature ids to values. This is important because almost all feature values will
# be 0 for most documents and it would be wasteful to save all in memory.

def features_to_ids(examples, feature_vocab):
    new_examples = sparse.lil_matrix((len(examples), len(feature_vocab)))
    for idx, example in enumerate(examples):
```

```
        for feat in example:
            if feat in feature_vocab:
                new_examples[idx, feature_vocab[feat]] = example[feat]
    return new_examples
```

```
# Evaluation pipeline for the Logistic Regression classifier.

def train_and_test(trainX, trainY, devX, devY, feature_functions, tokenizer):
    # Pre-process training documents.
    trainX_feat = docs2features(trainX, feature_functions, tokenizer)

    # Create vocabulary from features in training examples.
    feature_vocab = create_vocab(trainX_feat)
    print('Vocabulary size: %d' % len(feature_vocab))

    trainX_ids = features_to_ids(trainX_feat, feature_vocab)

    # Train LR model.
    lr_model = LogisticRegression(penalty='l2', C=1.0, solver='lbfgs', max_iter=1000)
    lr_model.fit(trainX_ids, trainY)

    # Pre-process test documents.
    devX_feat = docs2features(devX, feature_functions, tokenizer)
    devX_ids = features_to_ids(devX_feat, feature_vocab)
    # Test LR model.
    print('Accuracy: %.3f' % lr_model.score(devX_ids, devY))
```

```
from google.colab import drive
drive.mount('/content/drive')

import os
datapath = '/content/drive/MyDrive/data'

train_file = os.path.join(datapath, 'imdb_sentiment_train.txt')
dev_file = os.path.join(datapath, 'imdb_sentiment_dev.txt')

trainX, trainY = read_examples(train_file)
devX, devY = read_examples(dev_file)

features = [word_features]

# Evaluate LR model
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)
```

```
Mounted at /content/drive
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 28692
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
```

```
Processed 1500 examples into features
Accuracy: 0.839
```

Feature engineering

Evaluate LR model performance when adding positive and negative lexicon features. We will be using Bing Liu's sentiment lexicons from https://www.cs.uic.edu/~liub/FBS/sentiment-analysis.html

Read the positive and negative sentiment lexicons (5p)

There should be 2006 entries in the positive lexicon and 4783 entries in the positive lexicon.

```
# Read lexicons of positive and negative sentiment words
def read_lexicon(filename):
    lexicon = set()
    with open(filename, mode='r', encoding='ISO-8859-1') as file:
        for line in file:
            word = line.strip()
            if word and not word.startswith(";"):
                lexicon.add(word)
    return lexicon

# For Colab
lexicon_path = '/content/drive/MyDrive/data'

# Load lexicons
poslex_file = os.path.join(lexicon_path, 'positive-words.txt')
neglex_file = os.path.join(lexicon_path, 'negative-words.txt')

poslex = read_lexicon(poslex_file)
neglex = read_lexicon(neglex_file)

print(len(poslex), 'entries in the positive lexicon.')
print(len(neglex), 'entries in the negative lexicon.')

```

```
2006 entries in the positive lexicon.
4783 entries in the negative lexicon.
```

Use the lexicons to create two lexicon features (15p)

- A feature 'POSLEX' whose value indicates how many tokens belong to the positive lexicon.
- A feature 'NEGLEX' whose value indicates how many tokens belong to the negative lexicon.

```
def two_lexicon_features(tokens):
    feats = {'POSLEX': 0, 'NEGLEX': 0}
    # YOUR CODE HERE
    for token in tokens:
        if token in poslex:
            feats['POSLEX'] += 1
        if token in neglex:
            feats['NEGLEX'] += 1
    return feats
```

Evaluate the LR model using the two new lexicon features. Expected accuracy is around 83.8%.

```
# Specify features to use.
features = [word_features, two_lexicon_features]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)

```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
```

```
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 28694
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.839
```

Create a separate feature for each word that appears in each lexicon (20p)

- If a word from the positive lexicon (e.g. 'like') appears N times in the document (e.g. 5 times), add a positive lexicon feature 'POSLEX_word' for that word that is associated that value (e.g. {'POSLEX_like' : 5}.
- Similarly, if a word from the negative lexicon (e.g. 'dislike') appears N times in the document (e.g. 5 times), add a negative lexicon feature 'NEGLEX_word' for that word that is associated that value (e.g. {'NEGLEX_dislike' : 5}.

```
def lexicon_features(tokens):
    feats = {}
    # YOUR CODE HERE
    # Assume the positive and negative lexicons are available in poslex and neglex, respectively.
    for word in tokens:
        if word in poslex or word in neglex:
            feats['LEX_%s' % word] = 1
    return feats
```

Evaluate the LR model using the new per-lexicon word features. Expected accuracy is arpund 83.9%.

```
# Specify features to use.
features = [word_features, lexicon_features]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)

```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 31721
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
```

```
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.841
```

Add document length feature (5p)

Add a feature 'DOC_LEN' whose value is the natural logarithm of the document length (use *math.log* to compute logarithms).

```
import math
def length_feature(tokens):

    feats = {}
    if len(tokens) > 0:
        feats['DOC_LEN'] = math.log(len(tokens))
    return feats
```

```
negation_file = os.path.join(lexicon_path, 'negation_words.txt')

def read_negation_words(filename):
    negations = set()
    with open(filename, mode='r', encoding='utf-8') as file:
        for line in file:
            word = line.strip()
            if word:
                negations.add(word)
    return negations

negation_words = read_negation_words(negation_file)

```

```
def negation_features(tokens):
    feats = {}
    negated = False

    for token in tokens:
        if token in negation_words:
            negated = True
            continue
        if negated:
            feats[f"NEG_{token}"] = 1
            negated = False
    return feats

```

Evaluate the LR model using the new document length feature. Expected accuracy is around 84.0%.

```
# Specify features to use.
features = [word_features, lexicon_features, negation_features, length_feature]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)

```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 32960
Processed 100 examples into features
```

```
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.847
```

Add deictic features (15p)

Add a feature 'DEICTIC_COUNT' that counts the number of 1st and 2nd person pronouns in the document.

```
def deictic_feature(tokens):
    pronouns = set(('i', 'my', 'me', 'we', 'us', 'our', 'you', 'your'))
    count = 0

    # YOUR CODE HERE
    for token in tokens:
        if token.lower() in pronouns:
            count += 1

    return {'DEICTIC_COUNT': count}


# This is formatted as code
```

Evaluate the LR model using the deictic features. Expected accuracy is around 84.2%.

```
# Specify features to use.
features = [word_features, lexicon_features, length_feature, deictic_feature]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)

```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 31723
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.838
```

Let's try without the word features. Expected accuracy is around 80.4%.

```
# Specify features to use.
features = [lexicon_features, length_feature, deictic_feature]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)

```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 3031
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.799
```

Analysis ## (10p)

Include an analysis of the results that you obtained in the experiments above.

Error analysis: Do some basic error analysis where you try to explain the mistakes that the best model made and provide ideas for possible features that would alleviate these mistakes.

[BONUS] Interpretability (10p): From each class of features take 2 features that you think should be strongly correlated with the positive or negative label, and determine if the model learned a corresponding parameter that correctly expresses this correlation. For example, the feature 'WORD_loved' is expected to be very correlated with the positive label, as such the model should learn a corresponding large positive weight.

- *Hint: for this, you may consider using the `coef_` attribute of the LogisticRegression class.*

```
## Analysis (10p)

# #In this assignment, we explored various feature engineering techniques for sentiment classification using Logistic Regression.
# Starting with a baseline model using only word-level features, we achieved an accuracy of **83.9%**.
# Adding lexicon word indicators for positive and negative words improved the model to **84.1%**.
# We then introduced **negation features**, marking words like `NEG_amazing` following negation triggers such as "not" or "never".
# This provided the biggest boost, improving accuracy to **84.7%**. Incorporating **document length** using the
# natural log of the token count added robustness to review-length variation. Adding **deictic features**
#  (counting personal pronouns like “I”, “you”) helped capture personal tones and maintained high performance at
#  83.8%.
#Interestingly, even without using word-level features, the model still achieved **79.9%** accuracy using only lexicon,
#document length, and deictic features, showing the effectiveness of thoughtful linguistic signals.

---

# Error Analysis

```

```
# While the final model performed well, some errors persisted. Upon manual inspection, misclassified reviews often fell into two main ca

# - **Sarcasm and irony**: The model fails to capture ironic tone, e.g., *"This was the best worst movie ever."*
# - **Context dependency**: Words like *“dark”* or *“disturbing”* may be positive in the horror genre but are generally tagged as negati

# These suggest that potential improvements could include:
# - Adding **part-of-speech (POS)** tags to distinguish adjectives from other word uses.
# - Detecting **syntactic patterns** or **multi-word expressions**.
# - Incorporating **genre/context awareness** as a categorical feature.


```

```
#BONUS: Interpretability (10p)

# To evaluate interpretability, we inspected the `coef_` attribute of the trained `LogisticRegression`
#model using the final feature set. We expected words like `WORD_loved` and `POSLEX` to have **strong positive weights**,
#and words like `WORD_horrible` or `NEGLEX` to have **strong negative weights**.
# Here is a snippet to inspect weights:

import numpy as np

vocab = create_vocab(docs2features(trainX, features, spacy_tokenizer))
model = LogisticRegression(max_iter=1000)
X_train = features_to_ids(docs2features(trainX, features, spacy_tokenizer), vocab)
model.fit(X_train, trainY)

# Feature weights
coef = model.coef_[0]
sorted_weights = sorted(zip(vocab.keys(), coef), key=lambda x: x[1], reverse=True)

# Top positive and negative indicators
print("Top positive features:")
for feat, weight in sorted_weights[:10]:
    print(feat, weight)

print("\nTop negative features:")
for feat, weight in sorted_weights[-10:]:
    print(feat, weight)

```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Top positive features:
LEX_perfect 1.5019657405495201
LEX_wonderfully 1.3799387691730272
LEX_excellent 1.3171552629060146
LEX_winning 1.2758913715726976
LEX_great 1.2287981348014758
LEX_impossible 1.210942701636703
LEX_superb 1.2046724512562463
LEX_realistic 1.1863288807293526
LEX_fantastic 1.1615002169350614
LEX_tragic 1.118886291443394
```

```
Top negative features:
LEX_horrible -1.035965524359237
LEX_poorly -1.1029103326625567
LEX_unbelievable -1.1197159856672145
LEX_smart -1.1324464518147617
LEX_bad -1.14278560679735
LEX_boring -1.165208145300433
LEX_mediocre -1.188618886449251
LEX_worst -1.7392546380163176
LEX_terrible -1.984741091775254
LEX_waste -2.3415121524411875
```